Diffusion Language Models and Blockwise Decoding

Deep Dive into SGLang — Chapter 18

Yin Yang

Foundation Books Project

Where this chapter sits

Autoregressive serving appends *one* token per step. This chapter serves a model family that refines a whole masked block instead.

- A diffusion language model (dLLM) runs the reverse of masked discrete diffusion: iteratively *unmask* a token block.
- The runtime is still token serving — request rows, token ids, positions, logits, KV cache, streamed output.
- Only the *unit of work* changes: a request-owned block, not a single next-token row.

Goal: understand how SGLang joins block refinement to the existing serving contracts without pretending a block step is ordinary decode.

Roadmap

The blockwise generation problem

Phases, batches, and the forward mode

Unmasking, logits, and output

Boundaries and evidence

The blockwise generation problem

One request owns a mutable block

A request exposes B_{blk} masked positions, scores the whole block, accepts or edits some positions, then publishes tokens.

Passes may accept a different number of positions — but the scheduler and output path still see one request-owned block at fixed positions $o_r, \ldots, o_r + B_{\mathrm{blk}} - 1$.

Autoregressive decode vs. dLLM blockwise decode

Serving question	Autoregressive decode	dLLM blockwise decode
What is scheduled?	one next-token row	one block slice, up to B_{blk} rows
What is scored?	the next-token hidden state	full logits for every block row
What changes per pass?	one token commits	zero or more masked/editable positions
Mode name	`DECODE`	`DLLM_EXTEND`

Still token generation — but the executable unit is a request-owned block, not a single row.

Phases, batches, and the forward mode

Request phases: a block condition, not a request age

A dLLM request is *incoming* or *staging*, and *prefill* or *decode*.

- **Incoming / staging**: just entered the manager vs. carried from a prior round.
- **Prefill / decode**: prefill = active block has *no* masks; decode = at least one μ remains.

Prefill/decode is decided by the mask test on the active block — a short prompt whose first block still has holes is incoming-*decode*, even brand new.

The forward mode: DLLM_EXTEND

A dLLM step is executed as a `ForwardMode.DLLM_EXTEND` batch.

- Counted by `is_extend()` — assembled through extend-style metadata.
- Counted by `is_cuda_graph()` — fixed block-size shape can replay.
- Its own `is_dllm_extend()` predicate selects the full-block logits path.

Positions come from the block offset, not the live sequence length:

$$\text{positions}(r) = [\, o_r,\ o_r + 1,\ \ldots,\ o_r + B_{\text{blk}} - 1 \,].$$

Mask tokens are real rows — with positions, attention, and logits — not missing entries.

From concept to code: overriding positions

`ForwardBatch.init_new` builds block-offset positions when the batch has a dLLM config.

from `python/sglang/srt/model_executor/forward_batch_info.py`

```
# Override the positions with diffusion LLM or spec_info
if batch.dllm_config is not None:
    block_size = batch.dllm_config.block_size
    ret.positions = torch.tensor(
        [ i
          for block_offset in batch.dllm_block_offsets
          for i in range(block_offset, block_offset + block_size) ],
        dtype=positions_dtype,
    ).to(device, non_blocking=True)
```

Every block gets $o_r, \ldots, o_r + B_{\text{blk}} - 1$ — so a resumed block reads the right rotary positions and cache slots.

Unmasking, logits, and output

Two policies over full logits

The worker calls the dLLM algorithm's `run` instead of ordinary generation; it may run the model several times per scheduled batch.

`LowConfidence` mask-to-token only: accept masked positions above a confidence threshold, else accept the single most confident one; stop when no mask remains.

`JointThreshold` adds an *edit* phase: after masks clear, may rewrite non-prompt tokens under `edit_threshold`, bounded by `max_post_edit_steps`.

Confidence is the model's probability for its own argmax: $c_j = \text{softmax}(\ell_j)_{\hat{x}_j}$, with $\hat{x}_j = \arg\max_{v \in \mathcal{V}} \ell_j[v]$.

Why full logits, and why a final pass

- Any masked position may become the next accepted one; an edit may revisit a filled one. So the logits path returns full-block logits, not one next-token row.
- Output is a *list of variable-length token slices* — one per request — not a single next-token tensor.
- A final model pass makes cache-visible state match the committed block before the slice is published.

Once bytes are streamed, later block state cannot revise them — the mutable object is the uncommitted block, not the emitted stream.

From concept to code: the low-confidence loop

from `python/sglang/srt/dllm/algorithm/low_confidence.py`

```
x = torch.argmax(curr_logits, dim=-1)
p = torch.squeeze(torch.gather(
        F.softmax(curr_logits, dim=-1), dim=-1,
        index=torch.unsqueeze(x, -1)), -1)
confidence = torch.where(block_mask_index, p, -np.inf)
transfer_index = confidence > self.threshold
if transfer_index.sum().item() == 0:            # fallback: accept the best one
    _, select_index = torch.topk(confidence, k=1)
    transfer_index[select_index] = True
block_input_ids[transfer_index] = x[transfer_index]
```

Confidence is masked to μ positions; accept above threshold, else the single most confident masked position — exactly the definition.

Boundaries and evidence

Graph shape is fixed; refinement count is not

Capture records `DLLM_EXTEND` and sets `num_tokens_per_bs` to the block size:

$$(N_{\text{req}}, N_{\text{tok}}) = (N_{\text{req}},\ N_{\text{req}} \cdot B_{\text{blk}}).$$

- The replay surface is fixed (batch $\times$ block), but the policy may run a variable number of scoring passes.
- Server-arg handling narrows the path: disables overlap schedule, LoRA, disaggregation, hierarchical/LMCache, mixed chunked prefill; picks FlashInfer / Ascend / triton per hardware.

Each disabled feature would make block ownership or publication order ambiguous.

Evidence lives in a refinement account

A dLLM claim needs more than tokens/second. Record, per block:

$$\mathcal{A}_r = (B_{\mathrm{blk}}, F_r, U_r, E_r, O_r, m_r^{\mathrm{end}}, g_r^{\mathrm{run}}, c_r).$$

- forwards F_r, mask-to-token updates U_r, token edits E_r, emitted tokens O_r, final masks m_r^{end}, backend/graph setting, and whether the final pass changed cache state.
- Measured LLaDA2-mini rows (B_{blk}=32) exist for an offloaded one-request RTX 5090 — *not* a broad speedup claim.

Comparable runs fix model family, B_{blk}, backend, graph setting, and algorithm.

What to carry forward

1. dLLM serving is **masked-discrete reverse refinement** over tokens — one request owns one mutable block.
2. Phases classify by the **mask test**; execution runs as `DLLM_EXTEND` with **block-offset positions**.
3. Policies (`LowConfidence`, `JointThreshold`) choose accepts/edits from **full logits**.
4. Invariants keep **rows, masks, logits, cache, and output slices** aligned to the original request.
5. dLLM is a **token-serving** chapter — separate from Volume II media diffusion.

A block, not a token: refine under a mask, publish under a contract.